

UNIVERSIDAD SIGLO 21

Licenciatura en Informatica

Modalidad EDH

TRABAJO PRACTICO 4

Computo paralelo en la nube con MPI y GitHub Codespaces

Materia: Algoritmos Concurrentes y Paralelos

Codigo Canvas: INF394 - 11805

Profesor titular: Eduardo Enrique Piray

Alumno: Rodriguez, Daniel Sebastian

Legajo: VIN016869

DNI: 43.731.653

Fecha de entrega: 04/05/2026

1. Introduccion

El TP4 cierra el cuatrimestre y aplica los conceptos del modulo 4 sobre computo paralelo, esta vez usando la nube. La consigna es distinta a los TPs anteriores: el codigo ya esta hecho por el profesor en un repositorio de GitHub (https://github.com/edupiray/tp4_acyp), y lo que hay que hacer es hacer fork del repositorio, abrirlo en GitHub Codespaces, correr los tres programas que vienen adentro (multiplicacion de matrices, calculo de pi por el metodo de Monte Carlo y K-means para agrupar puntos), mirar las salidas y responder preguntas sobre como funciona cada uno.

La idea no es escribir codigo nuevo sino entender que hace cada parte del que ya esta y, sobre todo, por que se usa una funcion de MPI en cada lugar. MPI es el estandar para que varios procesos se pasen mensajes entre si, porque a diferencia de los hilos del TP2, los procesos no comparten memoria y necesitan comunicarse explicitamente.

2. Del fork al Codespace

Los pasos que segui son los que indica la consigna y el README del repositorio:

1. Inicie sesion en mi cuenta de GitHub.
2. Entre al repositorio https://github.com/edupiray/tp4_acyp y toque el boton Fork (arriba a la derecha). GitHub copia el repositorio en mi cuenta.
3. En mi fork, toque el boton verde Code, fui a la pestania Codespaces y al boton + para crear uno nuevo en la rama main. Despues de menos de un minuto se abre Visual Studio Code dentro del navegador, con una terminal abajo y todo listo para compilar (Ubuntu 22.04, MPICH instalado, gcc 11.4.0).
4. En la terminal navegue a cada carpeta (ejemplos_base y desafios_ml) y corri los comandos que figuran en cada README.md.

Lo que arma el entorno es el archivo .devcontainer/devcontainer.json del repositorio:

```
{
  "image": "ubuntu:22.04",
  "postCreateCommand": "apt update && apt install -y mpich libmpich-dev",
  "customizations": {
    "vscode": { "extensions": ["ms-vscode.cpptools"] }
  }
}
```

Esa configuracion alcanza para tener todo lo necesario sin instalar nada en la PC propia, que es una de las gracias de Codespaces. El repositorio trae tres carpetas: ejemplos_base con la multiplicacion de matrices y desafios_ml con los dos ejercicios de machine learning (Monte Carlo y K-means).

3. Producto de matrices

El programa multiplica dos matrices de 4×4 con numeros enteros aleatorios entre 0 y 9 (la constante N esta fijada en 4 dentro del codigo). La idea, traducida a pasos: el proceso 0 arma A y B, despues se reparten las filas de A entre los procesos con MPI_Scatter (una fila a cada uno, porque hay 4 filas y se corre con 4 procesos), se envia la matriz B entera a todos con MPI_Bcast, cada proceso multiplica su fila por B y obtiene una fila de la matriz resultado C, y al final MPI_Gather junta todas esas filas en el proceso 0 para armar la matriz C completa.

En la terminal del Codespace corri los dos comandos que figuran en el README.md de ejemplos_base:

```
# Version C
$ mpicc matmul.c -o matmul_c && mpirun -np 4 ./matmul_c
```

```
# Version C++
$ mpic++ matmul.cpp -o matmul_cpp && mpirun -np 4 ./matmul_cpp
```

Salida observada (recortada) para la version C, np=4:

```
Matriz A generada por proceso 0:
7 4 9 2
7 9 5 6
2 6 0 7
5 9 4 6
Matriz B generada por proceso 0:
1 7 5 3
3 2 9 5
9 2 3 3
1 2 6 0
Proceso 0 - Fila recibida de A: 7 4 9 2
Proceso 1 - Fila recibida de A: 7 9 5 6
Proceso 2 - Fila recibida de A: 2 6 0 7
Proceso 3 - Fila recibida de A: 5 9 4 6
Proceso 0 - Resultado parcial: 102 79 110 68
Proceso 1 - Resultado parcial: 85 89 167 81
Proceso 2 - Resultado parcial: 27 40 106 36
Proceso 3 - Resultado parcial: 74 73 154 72
Matriz resultado C:
102 79 110 68
85 89 167 81
27 40 106 36
74 73 154 72
```

Cada proceso muestra la fila que le toco de A y la fila que calculo para C. Al final el proceso 0 imprime la matriz resultado completa. La version en C++ hace exactamente lo mismo (cambia la extension del archivo y se compila con mpic++ en lugar de mpicc) y devuelve otra matriz C distinta, pero solo porque las matrices A y B se generan con numeros aleatorios distintos cada vez (la semilla cambia con time(NULL)).

4. Monte Carlo para estimar pi

Este programa estima pi con el metodo de los dardos. La idea es imaginar un cuadrado de lado 1 con un cuarto de circulo de radio 1 adentro, tirar 1.000.000 de dardos al azar dentro del cuadrado y contar cuantos cayeron dentro del cuarto de circulo. Como el area del cuarto de circulo es $\pi/4$ y el area del cuadrado es 1, la proporcion entre aciertos y total, multiplicada por 4, da una estimacion de pi.

La paralelizacion es bastante natural: el proceso 0 le avisa a todos el total de dardos con MPI_Bcast, cada proceso tira su parte (por ejemplo 250.000 dardos cada uno si son 4 procesos) usando una semilla diferente time(NULL) + rank para que los numeros aleatorios sean distintos en cada uno, y al final MPI_Reduce suma todos los aciertos y los manda al proceso 0, que es el que calcula pi y lo imprime.

Salidas observadas para distintos numeros de procesos:

```
$ mpirun -np 1 ./montecarlo_c
Aciertos totales: 785787   pi estimado: 3.143148   Error: 0.001555

$ mpirun -np 2 ./montecarlo_c
Proceso 0: 392908   Proceso 1: 392989
Aciertos totales: 785897   pi estimado: 3.143588   Error: 0.001995

$ mpirun -np 4 ./montecarlo_c
Procesos 0..3: ~196.500 aciertos cada uno
Aciertos totales: 785729   pi estimado: 3.142916   Error: 0.001323

$ mpirun -np 8 ./montecarlo_c
Procesos 0..7: ~98.200 aciertos cada uno
Aciertos totales: 785807   pi estimado: 3.143228   Error: 0.001635
```

Lo primero que se ve es que cada proceso saca aproximadamente la misma cantidad de aciertos (porque le toca la misma cantidad de dardos), y que el total se mantiene siempre cerca de 785.000, que es justamente lo que se espera porque $\pi/4$ es alrededor del 78,5 % del area del cuadrado. El valor estimado de π queda en torno a 3,142 en todas las corridas, pero NO es identico entre una y otra porque cada proceso usa una semilla distinta y los dardos caen en lugares diferentes cada vez.

5. K-means clustering

El K-means es un algoritmo para agrupar puntos en grupos (clusters). Pensado como ejemplo facil: imaginar 1000 puntos sobre un mapa que se quieren agrupar en 3 barrios. El algoritmo elige tres centros al azar, asigna cada punto al centro mas cercano, despues recalcula cada centro como el promedio de los puntos de su barrio, y repite el proceso hasta que los centros dejan de moverse mucho (en el codigo del profe se fijan 20 iteraciones).

La paralelizacion va asi: el proceso 0 arma los puntos y elige los tres centros iniciales al azar, los manda a todos con MPI_Bcast y reparte los puntos con MPI_Scatter (250 puntos para cada proceso si se corre con 4). En cada iteracion, cada proceso ve a que centro queda mas cerca cada uno de sus puntos y va sumando las coordenadas por cluster. Despues MPI_Allreduce suma esas cuentas y esas sumas de todos los procesos y deja el resultado en TODOS (no solo en el proceso 0), el proceso 0 calcula los nuevos centros y los redistribuye con MPI_Bcast para que todos los usen en la siguiente vuelta.

Salida observada para np=4:

```
$ mpicc kmeans_mpi.c -o kmeans_c -lm
$ mpirun -np 4 ./kmeans_c
=== K-MEANS PARALELO CON MPI ===
Centros iniciales:
C0: (5.16, 6.97)
C1: (8.76, 2.92)
C2: (2.19, 1.69)
Centros finales despues de 20 iteraciones:
Cluster 0: (4.28, 7.97) - 350 puntos
Cluster 1: (8.04, 3.40) - 323 puntos
Cluster 2: (2.55, 2.70) - 327 puntos
```

Se ve como los centros se movieron desde la posicion aleatoria del principio hasta el centro real de cada grupo, y los 1000 puntos quedaron repartidos bastante parejo entre los tres clusters (350, 323 y 327). Con 2 procesos o con 5 procesos pasa lo mismo, lo unico que cambia son los puntos generados al inicio porque la semilla srand(time(NULL)) tira numeros distintos cada vez.

6. Respuestas a las preguntas del punto 7

Producto de matrices

- **Que funcion cumple MPI_Scatter en el codigo?** Reparte la matriz A entre los procesos: a cada proceso le entrega una fila distinta para que la multiplique.
- **Por que se usa MPI_Bcast para la matriz B?** Porque cada proceso necesita la matriz B entera para multiplicar su fila de A contra todas las columnas de B, asi que se le copia la misma B a todos.
- **Cuantas filas calcula cada proceso cuando se usan 4 procesos para una matriz 4x4?** Una fila cada uno, porque hay 4 filas y 4 procesos (4 dividido 4 es 1).
- **Que ocurre si se ejecuta con mas procesos que filas de la matriz?** Falla: con mas procesos que filas el reparto queda mal (cada proceso recibe menos de una fila), aparece basura en los datos y MPI termina con error (lo probe con 8 procesos y la matriz de 4 filas).

- **Por que solo el proceso 0 imprime el resultado final?** Porque MPI_Gather le manda todas las filas de C unicamente al proceso 0, asi que es el unico que tiene la matriz completa para imprimir.

Monte Carlo pi

- **Por que cada proceso necesita una semilla aleatoria diferente?** Porque si todos usan la misma semilla generan exactamente los mismos dardos en los mismos lugares, asi no se amplia la muestra. Por eso se usa time(NULL) + rank, para que cada proceso tire numeros distintos.

- **Que funcion MPI permite sumar los aciertos de todos los procesos?** MPI_Reduce con la operacion MPI_SUM. Suma los aciertos de cada proceso y deja el total en el proceso 0, que es el unico que necesita ese numero para calcular pi.

- **Si ejecutas con 1 y con 4 procesos, el valor de pi es exactamente igual? Por que?** No, da parecido pero no identico. En mi corrida me dio 3,143148 con 1 proceso y 3,142916 con 4. La diferencia viene de que con mas procesos hay mas semillas y los dardos caen en lugares distintos.

- **Como afecta el numero de procesos al tiempo de ejecucion?** Con mas procesos el tiempo deberia bajar, pero en mi prueba con un millon de dardos en una sola maquina la diferencia es minima porque MPI tarda mas en coordinar lo que ahorra en computo (1 y 4 procesos dieron tiempos casi iguales).

- **Podrias modificar el programa para que cada proceso lance distinta cantidad de dardos?** Si, se podria reemplazar la division pareja (total/size) por un arreglo con la cuota de cada proceso, o usar MPI_Scatterv. El MPI_Reduce final suma igual sin importar cuantos dardos tiro cada uno.

K-means

- **Que diferencia hay entre MPI_Reduce y MPI_Allreduce? Por que aqui usamos MPI_Allreduce?** MPI_Reduce deja el resultado solo en el proceso 0; MPI_Allreduce lo deja en todos. Aca lo necesitamos en todos porque cada proceso tiene que saber los nuevos centros para reasignar sus puntos en la siguiente vuelta.

- **Si el numero de procesos no divide exactamente a N_PUNTOS, como podrias distribuir los datos?** Cambiando MPI_Scatter por MPI_Scatterv, que permite mandarle a cada proceso una cantidad distinta de puntos en lugar de partes iguales.

- **Por que los centros iniciales se eligen aleatoriamente? Que riesgo existe?** Porque empezar al azar es facil y no requiere conocer los datos. El riesgo es que la eleccion sea mala (por ejemplo dos centros muy cerca) y el resultado quede atrapado en una solucion poco util.

- **Que papel cumple MPI_Bcast dentro del bucle principal?** Reparte los nuevos centros (que calcula el proceso 0) a todos los demas, para que en la siguiente vuelta del ciclo todos los procesos trabajen con los mismos centros.

- **Al aumentar MAX_ITER o la cantidad de puntos, como se comporta la relacion computo/comunicacion?** Al aumentar las iteraciones o los puntos, los procesos pasan mas tiempo calculando y proporcionalmente menos tiempo comunicandose, con lo cual la paralelizacion se vuelve mas eficiente.

7. Conclusion

Los tres ejercicios muestran tres maneras distintas de paralelizar con MPI. La multiplicacion de matrices es la mas tipica: el proceso 0 reparte el trabajo, todos calculan su parte y al final se junta el resultado en uno solo. Monte Carlo es mas simple porque cada proceso trabaja por su cuenta sin necesitar nada de los demas, y solo al final se suman los resultados. K-means es el

mas interesante porque mezcla los dos enfoques: cada proceso trabaja con sus puntos pero al final de cada vuelta TODOS necesitan saber los nuevos centros para seguir, y ahi aparece MPI_Allreduce.

Lo bueno de hacer el TP en GitHub Codespaces es que no hubo que instalar MPI ni configurar nada en mi PC, el devcontainer.json deja todo listo en un minuto. Eso permite enfocarse en entender por que cada funcion de MPI esta donde esta, en lugar de pelear con la instalacion. Las preguntas del punto 7 ayudan justamente a eso, a fijarse por que en un caso se usa Reduce y en otro Allreduce, por que se reparte una matriz y la otra se replica, etc.

Referencias

Piray, E. (2026). Repositorio tp4_acyp. https://github.com/edupiray/tp4_acyp

Universidad Siglo 21. (2026). Algoritmos Concurrentes y Paralelos: Lecturas del Modulo 4. Material de catedra (cursada Marzo-Mayo 2026).

MPICH Project. (2024). MPICH user documentation. <https://www.mpich.org/documentation/>